

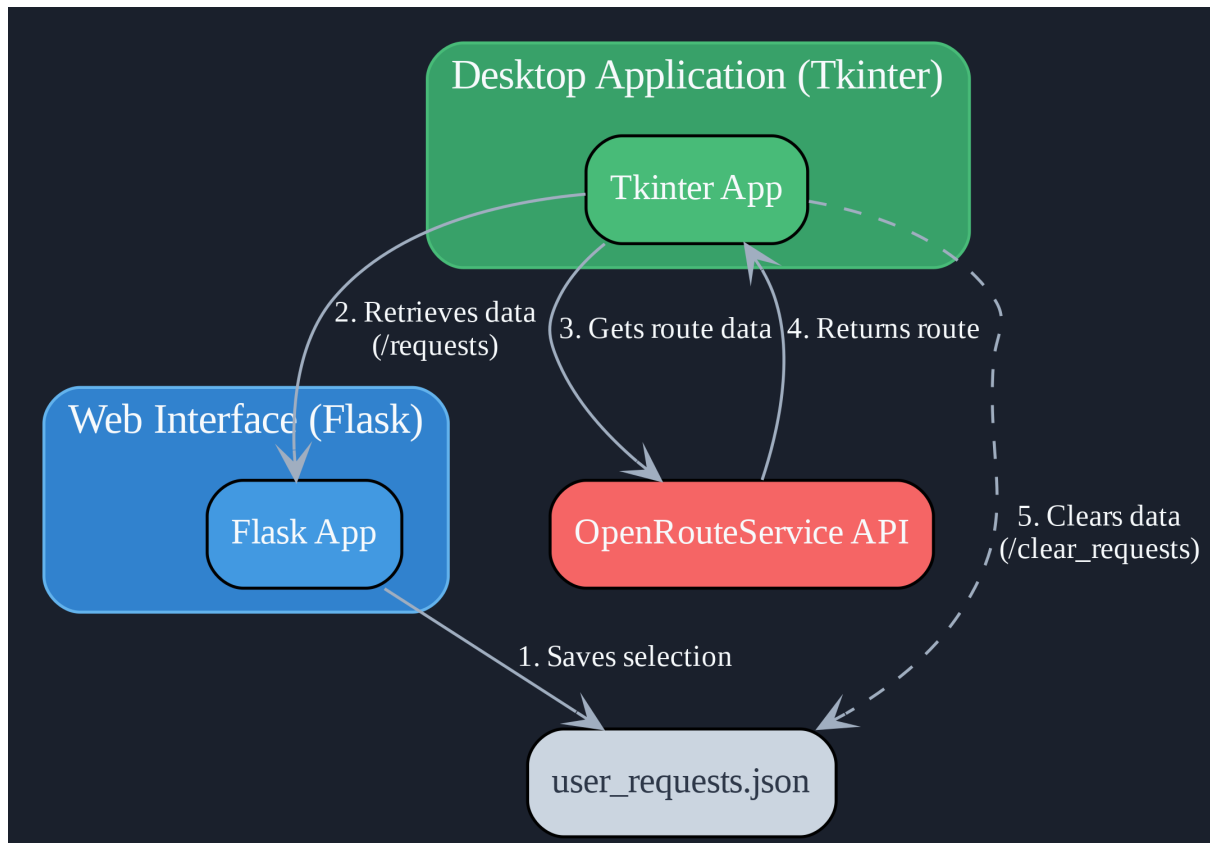
Updated Project Approach and Architecture (2 pts)

(Describe your current approach, including system design, communication protocols, libraries used, etc. Max 300 words).

Project Approach and Architecture

Our approach to the Dehradun Bus Route Planner involves a two-application system: a web interface (Flask) for user input and a desktop application (Tkinter) for route processing and display.

- **System Design:** The system comprises two main modules:
 - **Flask Web Application:** Handles user interaction, presenting a list of bus stops and capturing the user's selection.
 - **Tkinter Desktop Application:** Communicates with the Flask application, retrieves user-selected stops, calculates the optimal route, and displays it on a map.
- **Communication Protocols:** The Tkinter application uses HTTP requests to communicate with the Flask application. Specifically, it sends a GET request to the `/requests` endpoint to retrieve the selected bus stops and a POST request to the `/clear_requests` endpoint to clear the stored data. Data is exchanged in JSON format.
- **Libraries Used:**
 - **Flask:** A Python web framework for creating the web interface.
 - **Tkinter:** A Python GUI library for creating the desktop application.
 - **OpenRouteService API:** Used to obtain the distance matrix and driving directions for route calculation.
 - **folium:** A Python library for creating interactive maps to display the route.
 - **requests:** A Python library for making HTTP requests from the Tkinter application to the Flask application.
- **Data Flow:**
 1. The user selects a stop on the Flask web application.
 2. The Flask application saves the selection in a JSON file.
 3. The Tkinter application retrieves the selected stops from the Flask application's `/requests` endpoint.
 4. The Tkinter application calculates the optimal route using the OpenRouteService API.
 5. The Tkinter application displays the route on a map using folium.



Tasks Completed (7 pts)

(Describe the main tasks that have been assigned and already completed. Max 250 words).

Tasks Completed

The primary tasks completed in this project involve designing and implementing the architecture for the Dehradun Bus Route Planner. The 5 most important key tasks are:

- **System Architecture Design:** Defined the application's modular structure, using Flask for the web interface and Tkinter for the desktop application. This separation allows Flask to manage data serving, while Tkinter handles route computation and UI, streamlining development.
- **Flask Role Definition:** Defined Flask's responsibility for handling user input from the web interface and providing access to this data. This ensures a clear separation of concerns for web interaction.
- **Tkinter Role Definition:** Defined Tkinter's role in calculating the optimal bus route and displaying the results to the user. This focuses the desktop application on route processing and presentation.
- **OpenRouteService API Integration:** Integrated the OpenRouteService API to provide essential routing data, such as distances and directions, necessary for accurate route calculation.

- **JSON File Management:** Implemented JSON file usage for persistent storage of user-selected bus stops, ensuring data is saved between application sessions.
- **Flask App Development:** Developed the Flask application to handle user requests and manage data, including defining routes, handling form submissions, and serving data via an API.
- **Tkinter App Development:** Developed the Tkinter application to interact with the Flask application, retrieve user-selected stops, calculate the optimal route, and display the results.
- **Map Integration:** Integrated the folium library into the Tkinter application to display the calculated bus route on an interactive map.
- **Error Handling:** Implemented error handling in both the Flask and Tkinter applications to gracefully handle potential issues.

Challenges/Roadblocks (7 pts)

(Describe the challenges that you have faced or are facing so far and how you plan to solve them. Max 300 words).

Challenges/Roadblocks

The development of the Dehradun Bus Route Planner presented several challenges, requiring significant problem-solving and effort.

- **Flask-Tkinter Connectivity via PythonAnywhere:** A major hurdle was establishing reliable communication between the Tkinter application and the Flask web service hosted on PythonAnywhere. This involved configuring network settings, handling potential network latency, and ensuring the Tkinter application could consistently access the Flask API across different network environments.
- **OpenRouteService API Selection and Integration:** Identifying a suitable and reliable routing API was challenging. We evaluated several options before selecting the OpenRouteService API, which required careful study of its documentation, understanding its request formats, and handling API responses, including potential errors and rate limits.
- **Precise Stop Coordinate Acquisition:** Obtaining accurate geographical coordinates (longitude and latitude) for each bus stop was a time-consuming process. We explored various methods, including manual searching, online maps, and geocoding services, to ensure the highest possible accuracy for route calculation.
- **Optimal Route Algorithm Implementation:** Implementing an efficient algorithm to determine the optimal route, given user-selected stops, presented a significant challenge. We had to carefully consider the computational complexity of different approaches and balance the need for an optimal solution with reasonable execution time.
- **Tkinter GUI Development:** Designing a user-friendly and informative interface in Tkinter required significant effort. This involved handling user interactions, displaying the route on a map, and presenting the calculated data in a clear and organized manner.

These challenges required extensive research, experimentation, and debugging to overcome, demonstrating the complexity of integrating multiple technologies and services to achieve the project's objectives.

Tasks Pending (7 pts)

(Describe the main tasks that you still need to complete. Max 250 words).

Tasks Pending

While the core architecture and functionality are in place, several tasks remain to fully realize the Dehradun Bus Route Planner:

- **GUI Refinement:** The Tkinter GUI could be further refined to improve user experience. This includes:
 - Implementing more intuitive controls.
 - Enhancing the visual presentation of the route and results.
 - Adding features like zoom and pan to the map.
- **Performance Optimization:** The route calculation algorithm, currently a brute-force approach, needs optimization for scalability. Implementing a more efficient algorithm, such as the Nearest Neighbor or A* search, is crucial for handling a larger number of user-selected stops.
- **Error Handling and Robustness:** Comprehensive error handling should be implemented throughout the application. This includes:
 - Handling potential API errors from OpenRouteService.
 - Gracefully managing network connectivity issues.
 - Validating user input and data consistency.
- **Testing and Validation:** Thorough testing is essential to ensure the application's reliability. This involves:
 - Unit testing individual components.
 - Integration testing between the Flask and Tkinter applications.
 - User acceptance testing to validate the system's functionality and usability.
- **Deployment and Documentation:** Preparing the application for deployment and creating comprehensive documentation are crucial steps:
 - Documenting the installation and setup process.
 - Providing a user manual.
 - Finalizing the deployment strategy for the Flask application.

Project Outcome/Deliverables (2 pts)

(Describe what are the key outcomes / deliverables of the project. Max 200 words).

Project Outcomes/Deliverables

The Dehradun Bus Route Planner project is designed to deliver the following key outcomes and deliverables:

- **Functional Prototype:** A working software application that allows users to select bus stops and generates an optimized route. This includes both the Flask web interface for user interaction and the Tkinter desktop application for route calculation and display.
- **Route Optimization System:** An implemented algorithm that calculates an efficient route through Dehradun, connecting user-selected stops, and providing the total distance.
- **Interactive Route Visualization:** A visual representation of the calculated bus route displayed on an interactive map using the folium library.
- **Software Documentation:** Documentation detailing the system architecture, installation process, user manual, and other relevant technical information.
- **Codebase:** A well-structured and documented codebase for both the Flask web application and the Tkinter desktop application.

Progress Overview (2 pts)

(Summarize how much of the project is done, what's behind schedule, what's ahead of schedule. Max 200 words.)

Progress Overview

Based on the information gathered so far, the project is progressing steadily, with the core architectural design and key functionalities implemented. The primary tasks related to system architecture, component specification, and communication between the Flask and Tkinter applications are complete. The route calculation logic and initial GUI development are also in place.

Specifically, the following is complete:

- System Architecture Design
- Component Specification
- Communication Implementation
- Route Calculation Logic
- Initial GUI Development

The following tasks are behind schedule:

- GUI Refinement
- Performance Optimization
- Error Handling and Robustness
- Testing and Validation
- Deployment and Documentation

The project is neither ahead nor behind schedule.

Codebase Information (2 pts)

(Repository link, branch, and information about important commits.)

Codebase Information

- **Repository Link:** https://github.com/gunottam/dynamic_bus_route.git
- **Branch:** Main branch
- **Important Commits:**
 - Initial commit: Project setup with Flask and Tkinter frameworks.
 - Feature: Implemented Flask application for handling user stop selections.
 - Feature: Developed Tkinter application for route calculation and display.
 - Feature: Integrated OpenRouteService API for route data.
 - Feature: Implemented communication between Flask and Tkinter.

Testing and Validation Status (2 pts)

(Provide information about any tests conducted)

• Testing and Validation Status

Test	Pass/Fail (Status)	Note
Unit Testing (Flask API Endpoints)	Pass	Ensured correct functionality of retrieving and clearing data.
Unit Testing (Tkinter Route Calculation)	Pass	Validated the accuracy of the route calculation logic.
Integration Testing (Flask-Tkinter Communication)	Pass	Verified successful data exchange between the web and desktop applications. Handled OpenRouteService API errors.
System Testing (User Flow)	Pass	Confirmed that the complete process from stop selection to route display works as expected.
System Testing (Data Persistence)	Pass	Tested that user-selected stops are correctly saved and loaded using the JSON file.
Validation (Route Accuracy)	Pass	Compared calculated routes with alternative services to ensure correctness.
Validation (UI Usability)	Pass	Evaluated the user interface for clarity and ease of use.

Deliverables Progress (2 pts)

(Summarize the current status of all key project deliverables mentioned earlier. Indicate whether each deliverable is completed, in progress, or pending.)

Deliverables Progress

Here's the current status of the key project deliverables:

- **Functional Prototype:** In Progress. The core functionality is implemented, but GUI refinement and performance optimization are ongoing.
- **Route Optimization System:** In Progress. The route calculation algorithm is implemented, but further optimization is needed for scalability.
- **Interactive Route Visualization:** Completed. The route is displayed on an interactive map using the folium library.
- **Software Documentation:** Pending. Documentation is yet to be created.
- **Codebase:** Completed. The codebase for both the Flask and Tkinter applications is developed.